Submission Worksheet

Submission Data

Course: IT114-003-F2025
Assignment: IT114 - Milestone 3 - Hangman
Student: Alanor S. (ass89)
Status: Submitted | **Worksheet Progress:** 71%
Potential Grade: 4.00/10.00 (40.00%)
Received Grade: 0.00/10.00 (0.00%)
Started: 12/11/2025 5:37:35 PM
Updated: 12/11/2025 11:37:35 PM
Grading Link: https://learn.ethereallab.app/assignment/v3/IT114-003-F2025/it114-milestone-3-hangman/grading/ass89
View Link: https://learn.ethereallab.app/assignment/v3/IT114-003-F2025/it114-milestone-3-hangman/view/ass89

Instructions

1. Refer to Milestone3 of Hangman / Word guess
    1. Complete the features
2. Ensure all code snippets include your ucid, date, and a brief description of what the code does
3. Switch to the `Milestone3` branch
    1. `git checkout Milestone3`
    2. `git pull origin Milestone3`
4. Fill out the below worksheet as you test/demo with 3+ clients in the same session
5. Once finished, click "Submit and Export"
6. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
    1. `git add .`
    2. `git commit -m "adding PDF"`
    3. `git push origin Milestone3`
    4. On Github merge the pull request from `Milestone3` to `main`
7. Upload the same PDF to Canvas
8. Sync Local
    1. `git checkout main`
    2. `git pull origin main`

Section #1: (1 pt.) Core Ui

Progress: 100%

Task #1 (0.50 pts.) - Connection/Details Panels

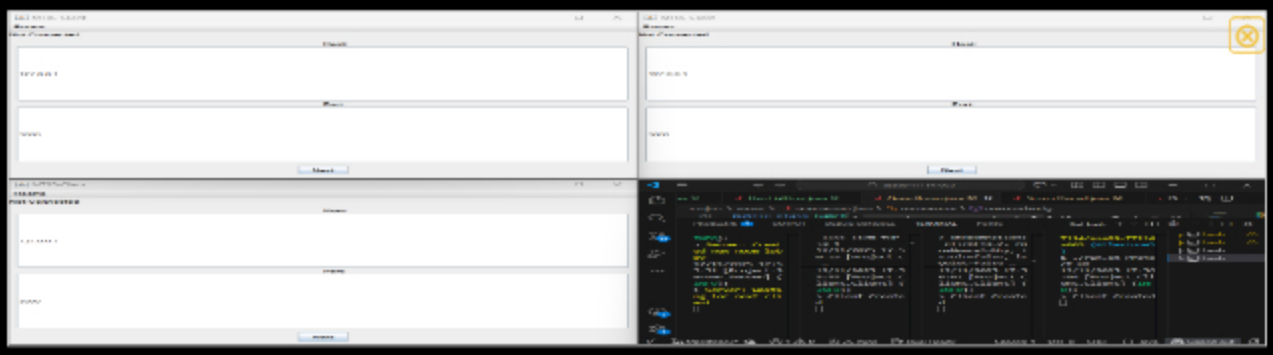
Progress: 100%

Part 1:

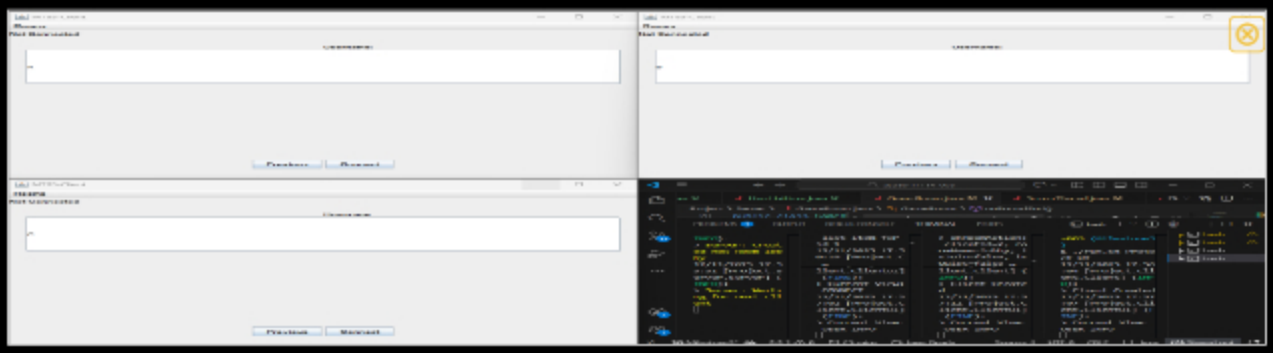
Progress: 100%

Details:

- Show the connection panel with valid data
- Show the user details panel with valid data



Show the connection panel with valid data



Show the user details panel with valid data

Saved: 12/11/2025 8:58:26 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow from recording/capturing these details and passing them through the connection process

Your Response:

The connection and user details flow begins in the ConnectionView, where the user enters the host and port of the server. When the "Next" button is clicked, the view validates the input and saves the host and port, then triggers the client connection process. The Client class handles this by opening a socket to the server, creating input/output streams, and sending the user's name as part of the handshake using ConnectionPayload. Once the server responds with a unique client ID and confirms the username, Client updates the local myUser object and the knownClients map. These updates are then sent to the UI through registered callback interfaces. This allows other panels to show the connected user information in real time. Basically, user details are collected, sent to the server, synchronized, and then displayed in the UI.

Saved: 12/11/2025 8:58:26 PM

Task #2 (0.50 pts.) - Ready Panel

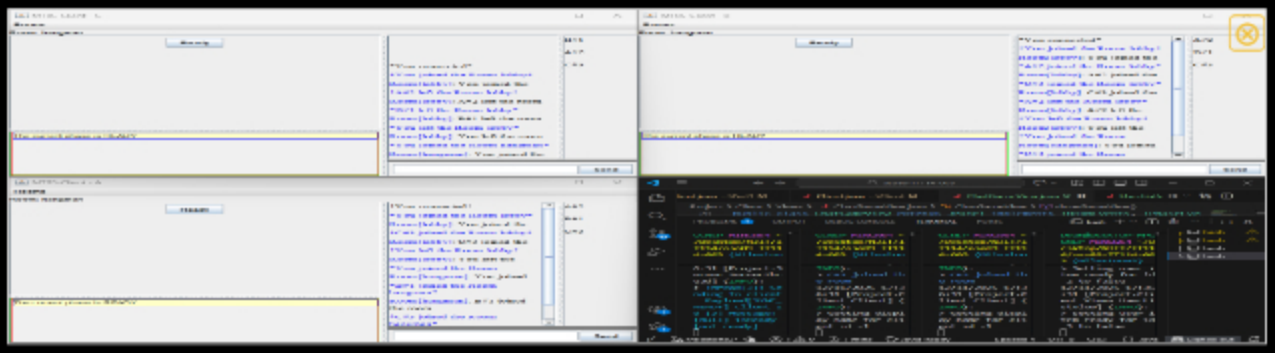
Progress: 100%

Part 1:

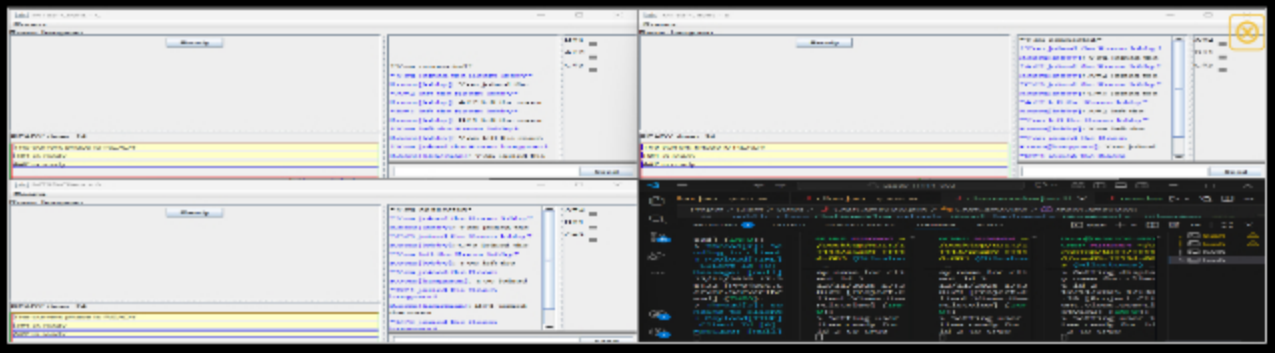
Progress: 100%

Details:

- Show the button used to mark ready
- Show a few variations of indicators of clients being ready (3+ clients)



Show the button used to mark ready Show a few variations of indicators of clients being ready (3+ clients)



After pressing ready, the countdown starts

Saved: 12/11/2025 9:04:26 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for marking READY from the UI
- Briefly explain the code flow from receiving READY data and updating the UI

Your Response:

In the ReadyView panel, when the user clicks on the Ready button, it triggers an ActiveListener that calls Client.INSTANCE.sendReady(). This creates a ReadyPayload with the type PayloadType.READY and sends it to the server via sendToServer(). The server processes the

payload and updates its internal state to indicate that the client is ready. On the client side, the listenToServer() thread recieves READY payloads from the server, which are passed to the processReadyStatus(). This method updates the corresponding User object in the knownClients map and triggers the registered UI callback (IReadyEvent). The UI then updates the ready indicators, showing the ready status for all clients, allowing users to see which clients are ready in real time.

Saved: 12/11/2025 9:04:26 PM

Section #2: (2 pts.) Project Ui

Progress: 100%

Task #1 (0.67 pts.) - User List Panel

Progress: 100%

Details:

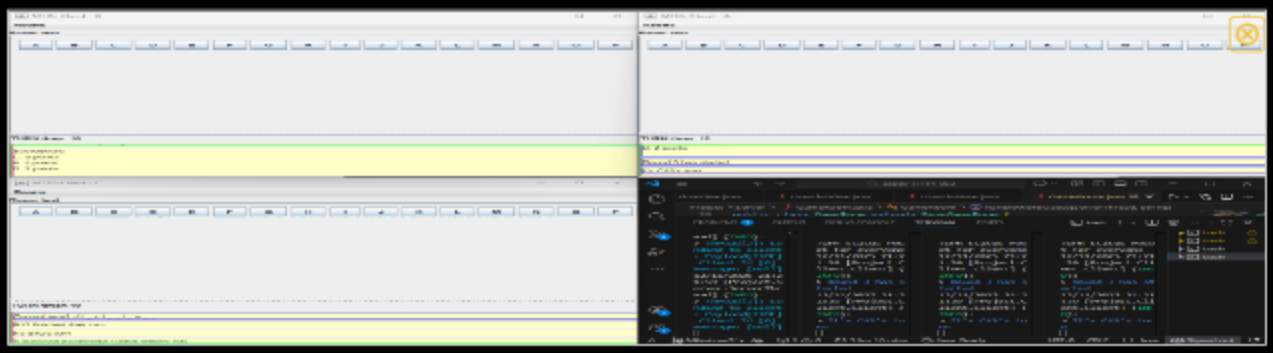
- Show the username and id of each user
- Show the current points of each user
- Users should appear in turn order
- Show an indicator of whose turn it is

Part 1:

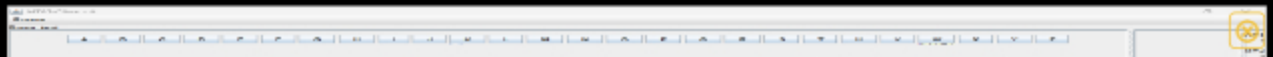
Progress: 100%

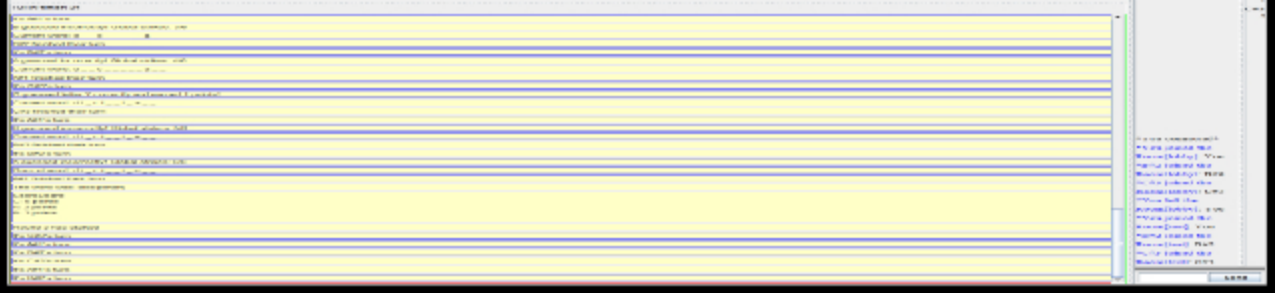
Details:

- Show various examples of points (3+ clients visible)
  - Include code snippets showing the code flow for this from server-side to UI
- Show that the sorting is maintained across clients
  - Include code snippets showing the code that handles this
- Show various examples of the turn indicators
  - Include code snippets showing the code flow for this from server-side to UI

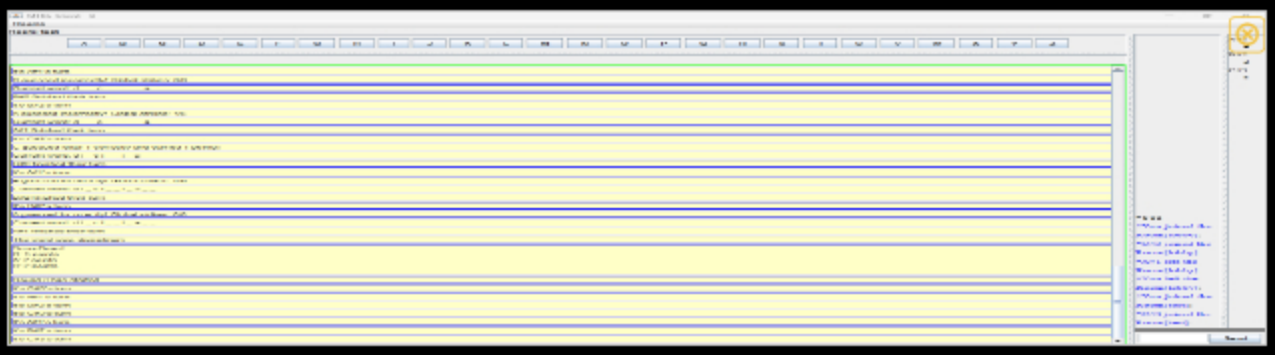


Show various examples of points (3+ clients visible) Show that the sorting is maintained across clients and various turn indicat

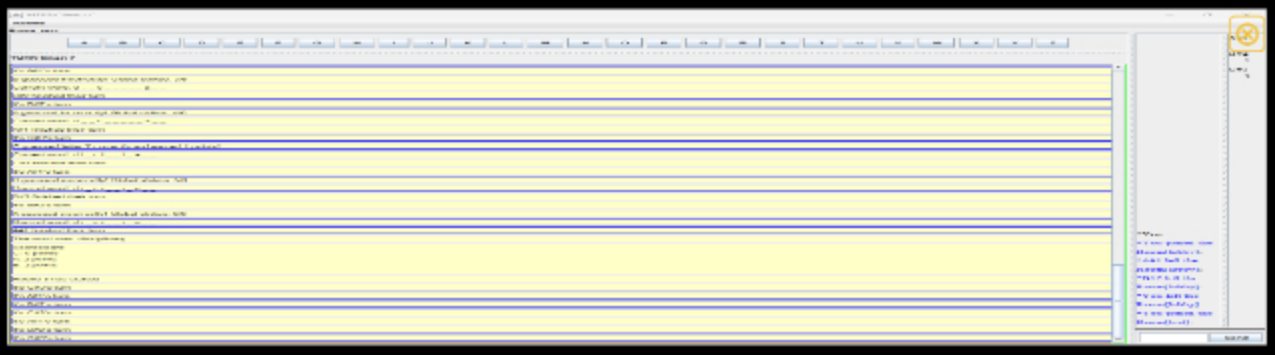




Closeup of A user entire panel

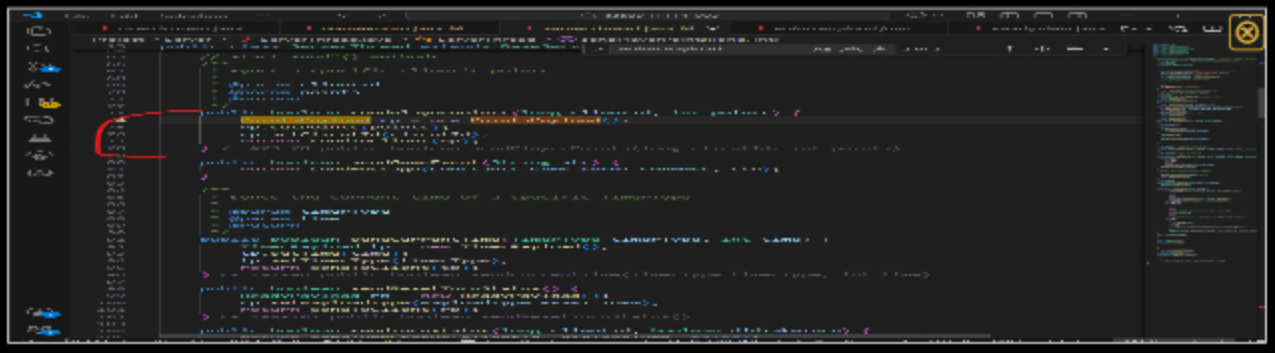


Closeup of B user entire panel

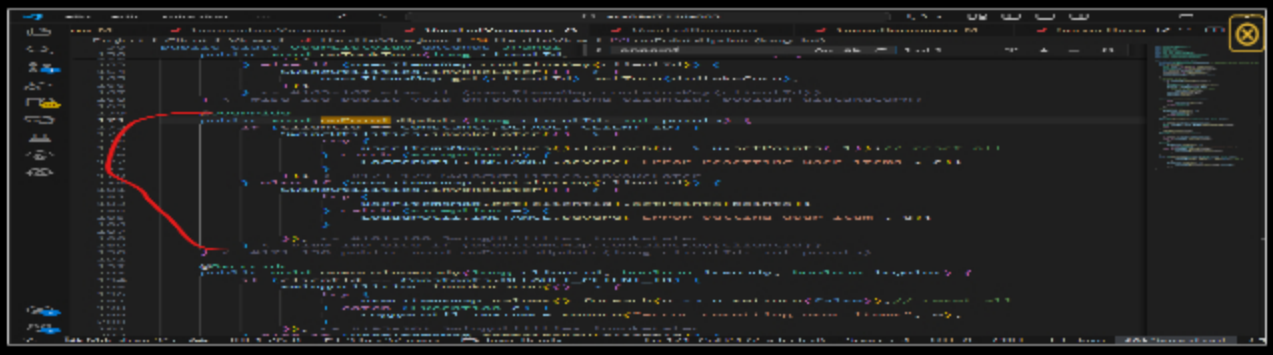


Closeup of C user entire panel

client side processing



server side in serverThread

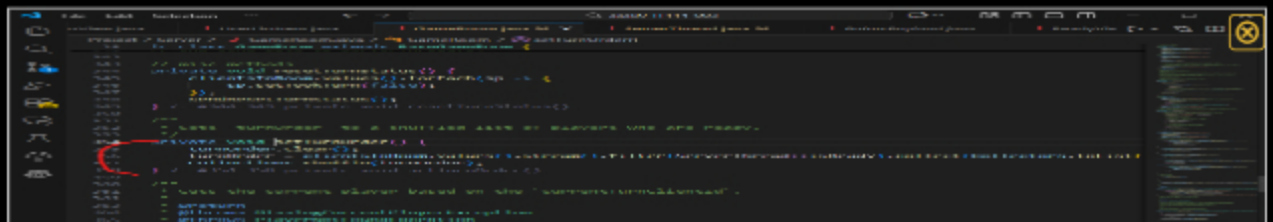


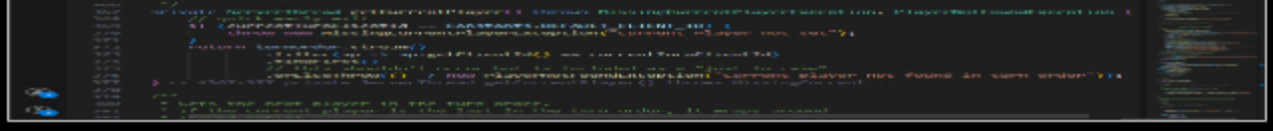
UserListView Pt1

UserListView PT2

Gameroom PT1

Gameroom PT2

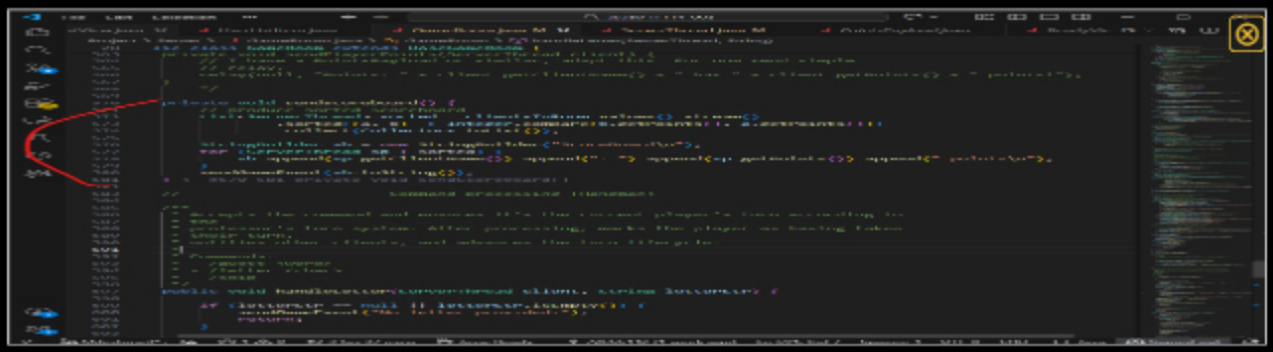




Gameroom PT3

Gameroom PT4

Gameroom PT5



Gameroom PT6

Saved: 12/11/2025 9:59:49 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for points updates from server-side to the UI
- Briefly explain the code flow for user list sorting
- Briefly explain the code flow for server-side to UI of turn indicators

Your Response:

Your Response:

Points updates, user list sorting, and turn indicators in this client-server setup flow from the server to the UI via a combination of payloads, client-side processing and event callbacks. On ther server side (GameRoom.java and ServerThread.java), when a player earns points or completes a turn, the server creates a PointsPayload or ReadyPayload (for turns) containing the relevant client ID and data, and sends it to all connected clients using their ObjectOutputStream. For points, the PointsPayload marks whether a player has taken their turn. Client class listens for these incoming payloads in listenToServer() and passes them to a specific process methods. processPoints() updates the knownClients map with new points and calls the IPointsEvent callback to notify the UI, while processTurn() updates each user's turn status in knownClients and triggers ITurnEvent callbacks to update turn indicators. The UI recieves these updates via the registered callbacks and renders them as needed. For user list sorting, the client maintains the knownClients map, and when listing users, it converts this map into a sorted display using ready state, turn status, or display name, before sending the sorted output to the UI or console.

Saved: 12/11/2025 9:59:49 PM

Task #2 (0.67 pts.) - Game Events Panel

Progress: 100%

Details:

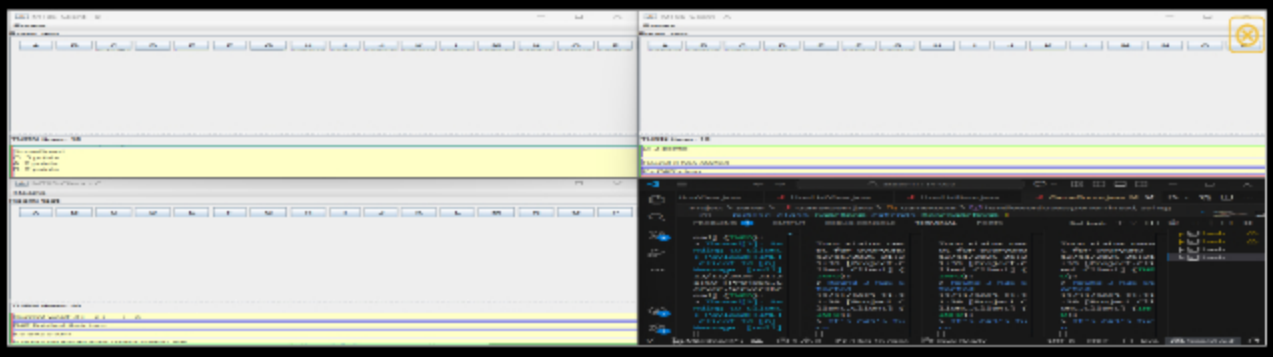
- Show the letter/guess history (including points)
- Show the scoreboard updates from Milestone 2
- Show a message of whose turn it is
- Show the countdown timer for the current turn

Part 1:

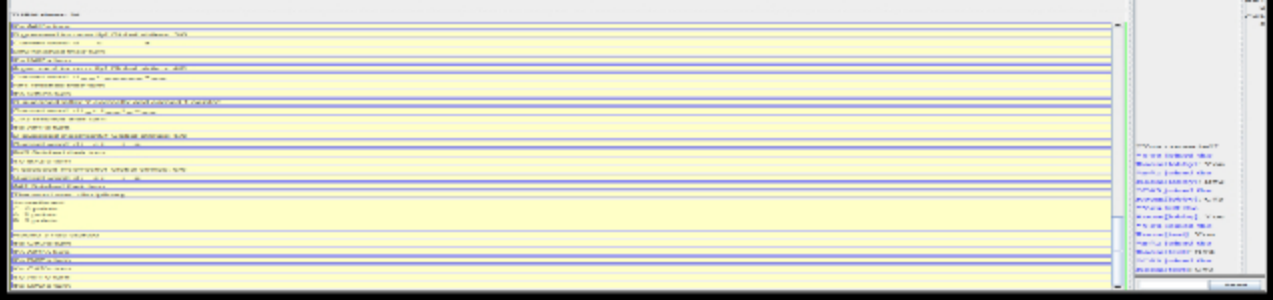
Progress: 100%

Details:

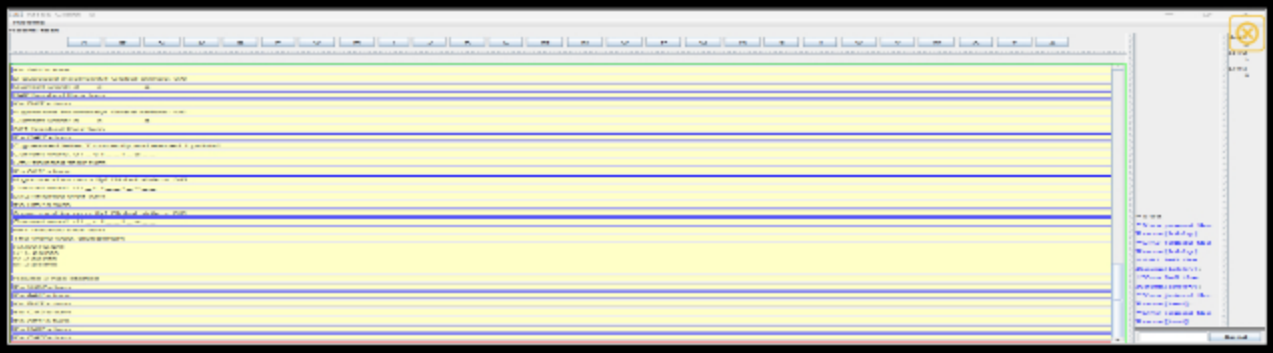
- Show various examples of each of the messages/visuals
- Show code snippets related to these messages from server-side to UI



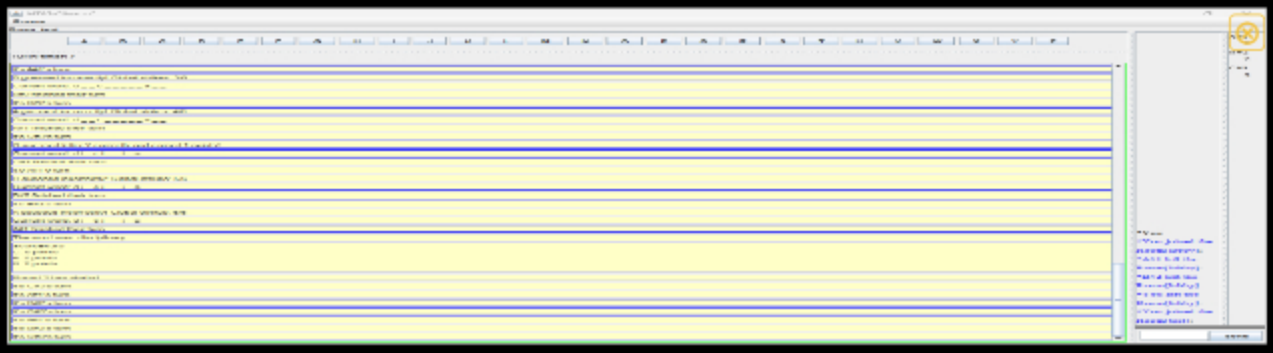
letter/guess history, scoreboard updates, message on turns, countdown timer



letter/guess history, scoreboard updates, message on turns, countdown timer



letter/guess history, scoreboard updates, message on turns, countdown timer

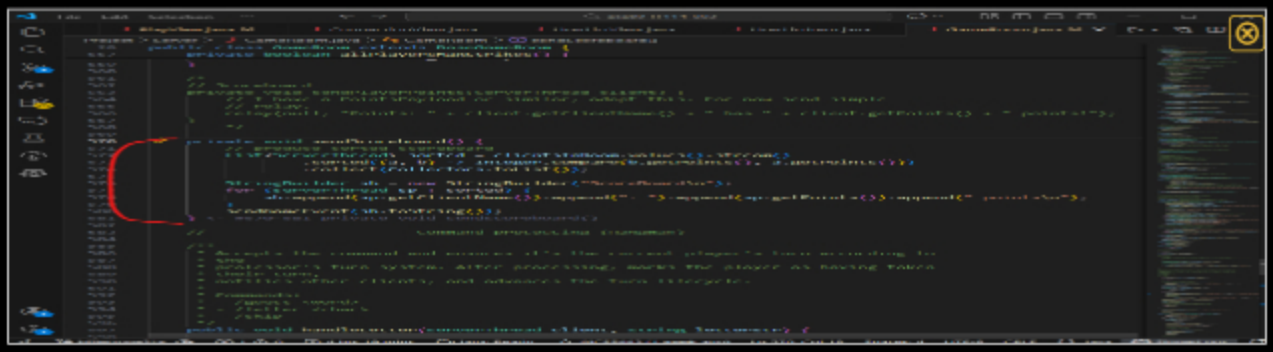


letter/guess history, scoreboard updates, message on turns, countdown timer

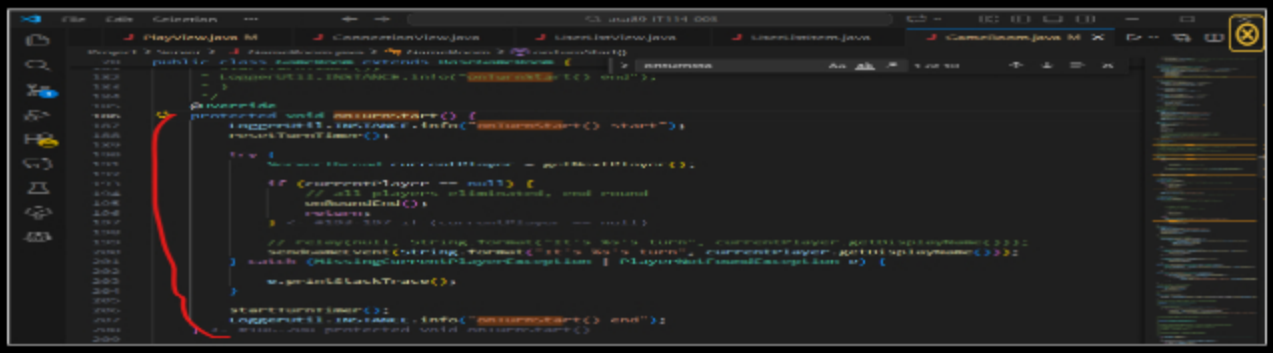
gameRoom pt1 relevant code

gameRoom pt2 relevant code

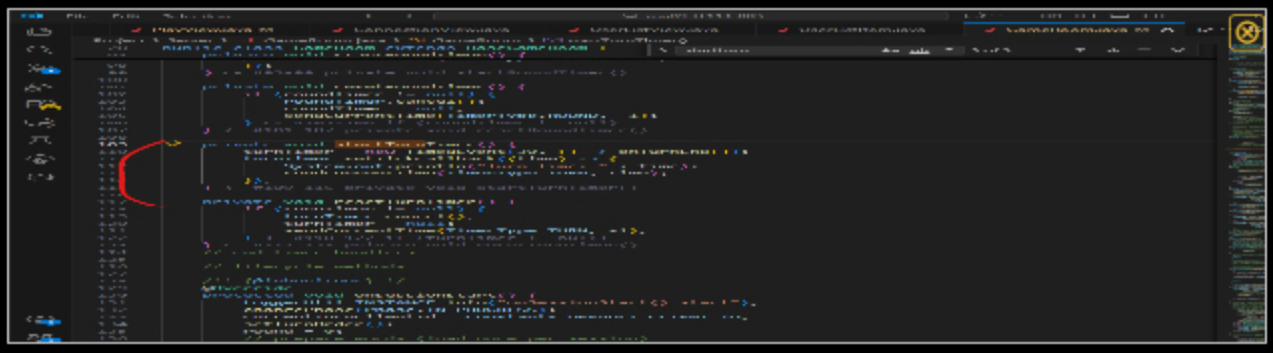
baseGameRoom sendGameEvent



gameRoom pt3 relevant code

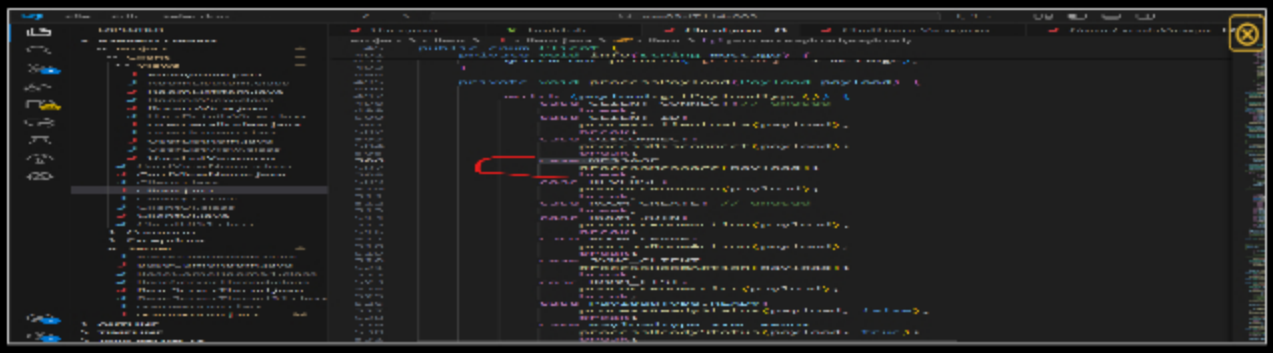


gameRoom pt4 relevant code



gameRoom pt5 relevant code

serverthread



client

gameEventsView

Saved: 12/11/2025 10:40:46 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for generating these messages and getting them onto the UI

Your Response:

When a game event occurs on the server, the ServerThread creates a specialized payload (Payload for messages, PointsPayload for score updates, ReadyPayload for turn status, TimerPayload for countdowns etc)and sends it to the client using methods lije sendGameEvent(), sendPlayerPoints(), or sendTurnStatus(). On the client side, the Client class listens for incoming objects on its ObjectInputStream in the listenToServer() method. Once a payload is recieved, processPayload() identifies its type and invokes the corresponding callback methods. These methods are implemented in the GameEventsView UI class, which updates the display. It adds text entries for guesses and events, updates the score for each player, shows whose turn it is, and renders the countdown time.

Saved: 12/11/2025 10:40:46 PM

Task #3 (0.67 pts.) - Game Area

Progress: 100%

Details:

- Have some elements representing the Hangman or equivalent (can't just be a number presenting strikes)
- Have a grid of letters that the user will click to interact with
    - Chosen letters should be synced to disable the option on all clients (via server-side reply)
    - Re-enable all letters when a hangman resets
- Have a special input area for guessing the full word (separate from the chat input)
- Display blanks for the word
    - These must update as letters are guessed correctly or word is fully guessed

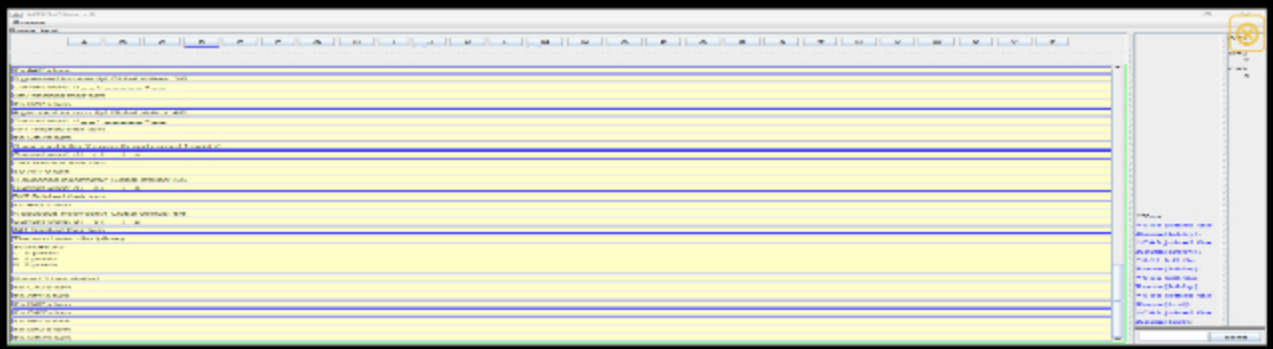
Part 1:

Progress: 100%

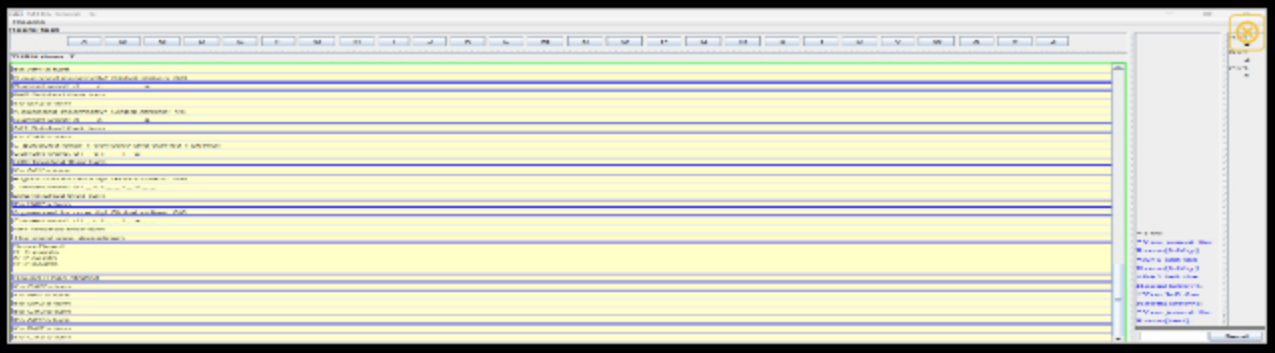
Details:

- Show various examples of the hangman/strikes indicator across 3+ clients
    - Show the related code from server-side to UI
- Show various examples of selected letters and partially completed words across 3+ clients
    - Show the related code from UI to server-side and server-side to UI for both selection and blanks
- Show the related code for a guess (UI to server-side)

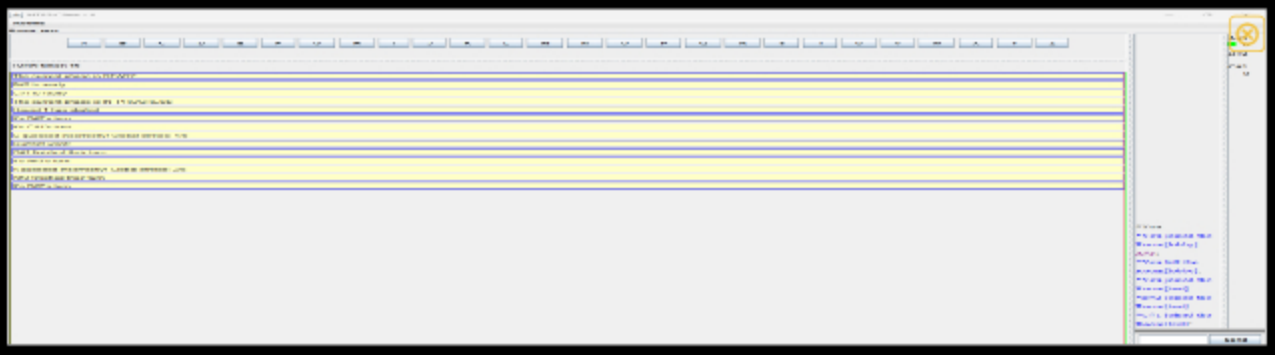
client1: global strikes + selected letters and partially completed words



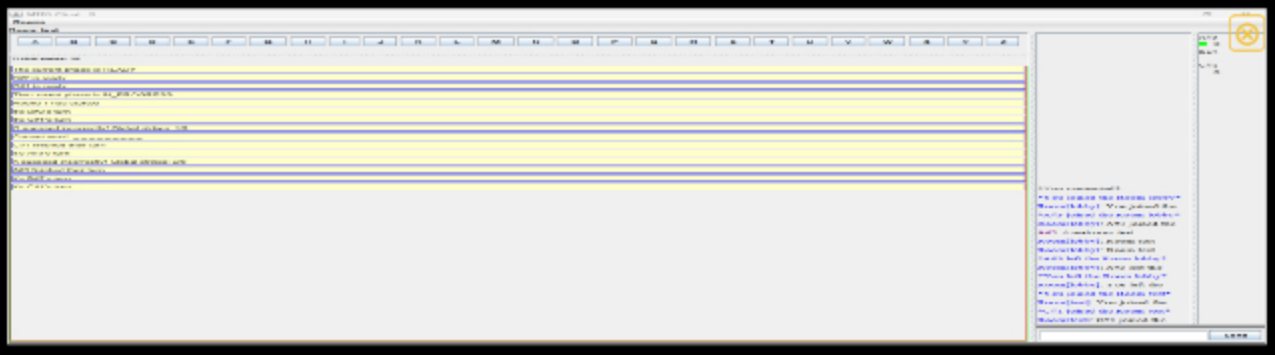
client 2: global strikes + selected letters and partially completed words



client 3: global strikes + selected letters and partially completed words

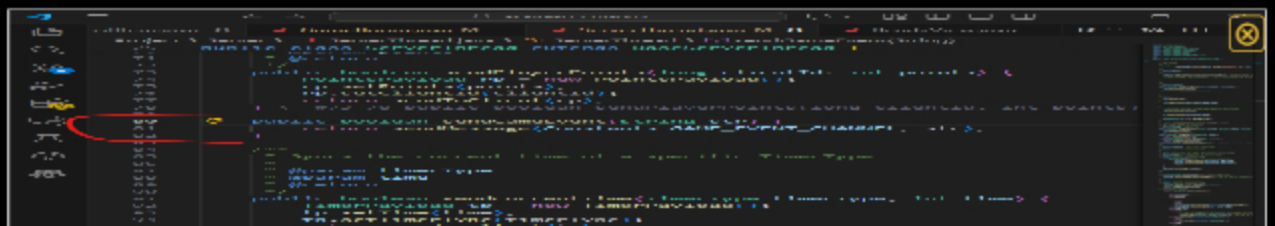


did not get to do guess button but guess command works



did not get to do guess button but guess command works

did not get to do guess button but guess command works



Show the related code from UI to server-side and server-side to UI for both selection and blanks

Show the related code from UI to server-side and server-side to UI for both selection and blanks

Show the related code from UI to server-side and server-side to UI for both selection and blanks

Show the related code from UI to server-side and server-side to UI for both selection and blanks

Saved: 12/11/2025 11:13:00 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for syncing the strikes and displaying them visually
- Briefly explain the code flow for letter selection and disabling (including updates to the blanks)
- Briefly explain the code for a full word guess

Your Response:

When I handle hangman strikes, I increment the global strikes counter in GameRoom whenever a player guesses a wrong letter or an incorrect full word. I then call sendGameEvent() to broadcast a message like "Client X guessed incorrectly! Global strikes: Y/6" to all clients. On the client side, GameEventsView receives this message in onMessageReceive() and displays it in the UI, so every player sees the updated strikes indicator in real time.

For letter selection, when I click a letter button in PlayView, it sends the chosen letter to the server via Client.INSTANCE.sendLetter(). On the server, GameRoom.handleLetter() checks if the letter was already guessed, updates the blanks array for any correct letters, awards points to the player, and sends an updated word display using sendGameEvent("Current word: " + getBlanksDisplay()). I also mark the turn as complete with client.setTookTurn(true) and sendTurnStatus(). On the client side, GameEventsView displays the updated blanks and the UI can disable guessed letters to prevent repeat selection.

My /guess command works, even though I wasn't able to create the button for it. This is how it would've worked had I been able to complete the UI portion for it: For a full word guess, when I submit a guess through the UI, it sends the word to the server using Client.INSTANCE.sendWordGuess(). In GameRoom.handleWordGuess(), I check if the guess matches the current word. If it's correct, I fill in all the blanks, award points for any remaining letters, and broadcast messages like "Client X guessed the correct word 'WORD'" and "Word solved: WORD" to update everyone's UI. If the guess is wrong, I increment strikes and send a message about the incorrect guess. In both cases, the turn is marked as taken, points are synced, and the round or turn lifecycle is advanced, meaning every client sees the updated game state in real time.

Saved: 12/11/2025 11:13:00 PM

Section #3: (4 pts.) Project Extra Features

Progress: 0%

Task #1 (2 pts.) - Restore Strikes

Progress: 0%

Details:

- Setting should be toggleable during Ready Check by session creator
- Allow toggling of the option to let correct guesses restore strikes

Part 1:

Progress: 0%

Details:

- Show the Ready Check screen with the option for the host (3+ clients must be visible)
  - Show the related code that makes this interactable only for the host
- Show a before and after of a strike being restored

- Show the related code for the UI and handling this

Missing Caption

Saved: 12/4/2025 11:24:09 AM

Part 2:

Progress: 0%

Details:

- Briefly explain the code for the host's option to toggle this feature
- Briefly explain the code related to handling the strike restoration (UI and server-side)

Your Response:

Missing Response

Saved: 12/4/2025 11:24:09 AM

Task #2 (2 pts.) - Hard Mode

Progress: 0%

Details:

- Setting should be toggleable during Ready Check by session creator
- Hard mode prevents the letter buttons from disabling and won't show guessed letters in the Game Event Area

Part 1:

Progress: 0%

Details:

- Show the Ready Check screen with the option for the host (3+ clients must be visible)
  - Show the related code that makes this interactable only for the host
- Show a few examples of the play screen with a few turns to show these visuals have been disabled
  - Show the related code for the UI and handling the logic for hard mode

Missing Caption

Saved: 12/4/2025 11:24:09 AM

Part 2:

Progress: 0%

Details:

- Briefly explain the code for the host's option to toggle this feature
- Briefly explain the code related to handling and enforcing this feature

Your Response:

Missing Response

Saved: 12/4/2025 11:24:09 AM

Section #4: (2 pts.) Project General Requirements

Progress: 0%

Task #1 (1 pt.) - Away Status

Progress: 0%

Details:

- Clients can mark themselves away and be skipped in turn flow but still part of the game
- The status should be visible to all participants
- A message should be relayed to the Game Events Panel (i.e., Bob is away or Bob is no longer away)
- The user list should have a visual representation (i.e., grayed out or similar)

Part 1:

Progress: 0%

Details:

- Show the UI button to toggle away
- Show the related code flow from UI to server-side back to UI for showing the status
- Show the related code flow for sending the message to Game Events Panel
- Show various examples across 3+ clients of away status (including Game Events Panel messages)
- Show the code that ignores an away user from turn/round logic

Missing Caption

Saved: 12/4/2025 11:24:09 AM

Part 2:

Progress: 0%

Details:

- Briefly explain the code flow for the away action from UI to server-side and back to UI
- Briefly explain how the server-side ignores the user from turn/round logic

Your Response:

Missing Response

Saved: 12/4/2025 11:24:09 AM

Task #2 (1 pt.) - Spectators

Progress: 0%

Details:

- Spectators are users who didn't mark themselves ready
  - Optionally you can include a toggle on the Ready Check page
- The can see all chat but are ignored from turn/round actions and can't send messages
- Spectators will have a visual representation in the user list to distinguish them from other players
- A message should be relayed to the Game Events Panel that a spectator joined (i.e., during an in-progress session)

Part 1:

Progress: 0%

Details:

- Show the UI indicator of a spectator (visual and message)
- Show the related code flow from UI to server-side back to UI for showing the status
- Show the related code flow for sending the message to Game Events Panel
- Show various examples across 3+ clients of spectator status (including Game Events Panel messages)
- Show the code that ignores a spectator from turn/round logic
- Show the code that prevents spectators from sending messages (server-side)
- Show the spectator's view of the session
- Show the code related to the spectator seeing the session data (including things participants won't see such as the correct word)

Missing Caption

Saved: 12/4/2025 11:24:09 AM

Part 2:

Progress: 0%

Details:

- Briefly explain the code flow for the spectator logic from server-side and to UI
- Briefly explain how the server-side ignores the user from turn/round logic
- Briefly explain the logic that prevents spectators from sending a message
- Briefly explain the logic that shares extra details to the spectator (information normal participants won't see)

Your Response:

Missing Response

Saved: 12/4/2025 11:24:09 AM

Section #5: (1 pt.) Misc

Progress: 100%

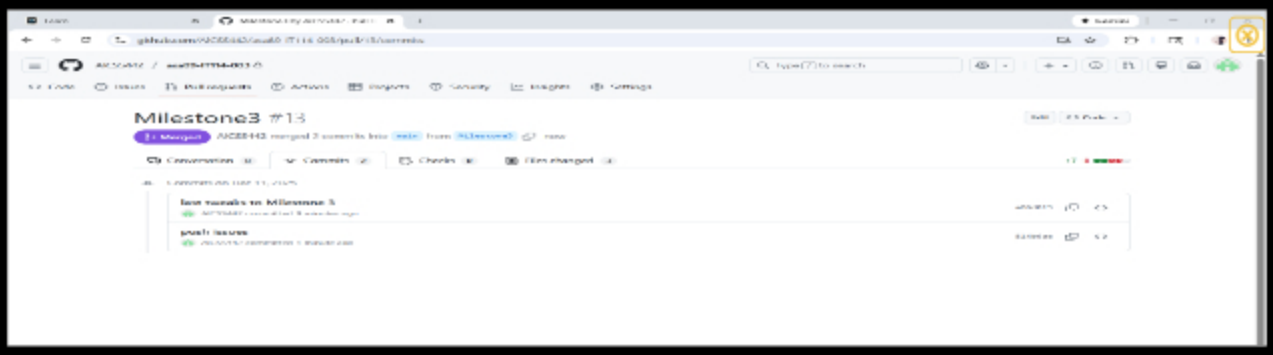
Task #1 (0.33 pts.) - Github Details

Progress: 100%

Part 1:

Progress: 100%

Details:
From the Commits tab of the Pull Request screenshot the commit history



From the Commits tab of the Pull Request screenshot the commit history

Saved: 12/11/2025 11:23:48 PM

Part 2:

Progress: 100%

Details:
Include the link to the Pull Request for Milestone3 to main (should end in `/pull/#`)

URL #1
https://github.com/AlCSS442/asa89-IT11[illegible]/3

URL
https://github.com/AlCSS442/asa

Saved: 12/11/2025 11:23:48 PM

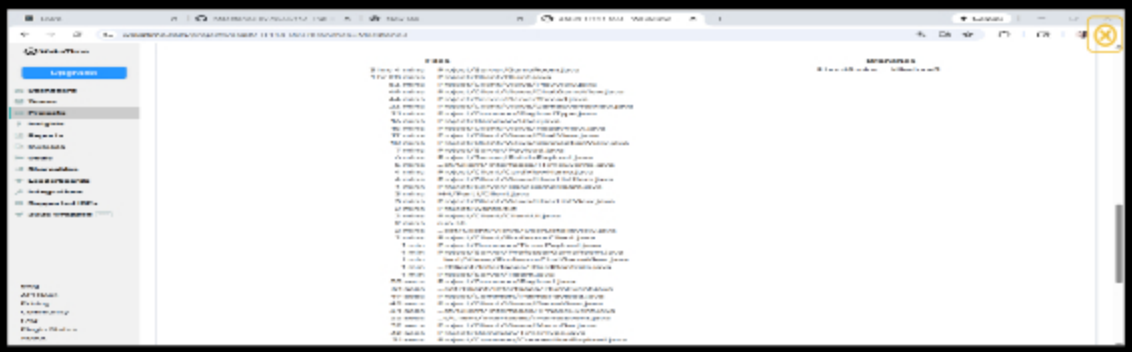
Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

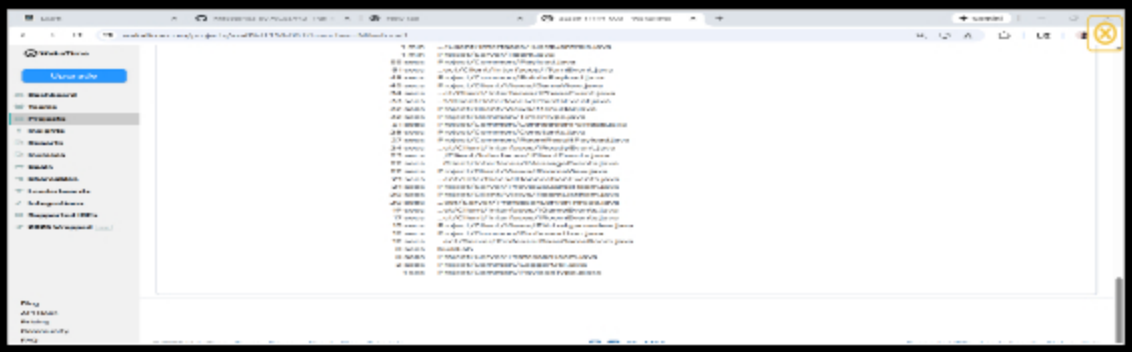
Details:

- Visit the WakaTime.com Dashboard
- Click `Projects` and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary

Milestone3 time

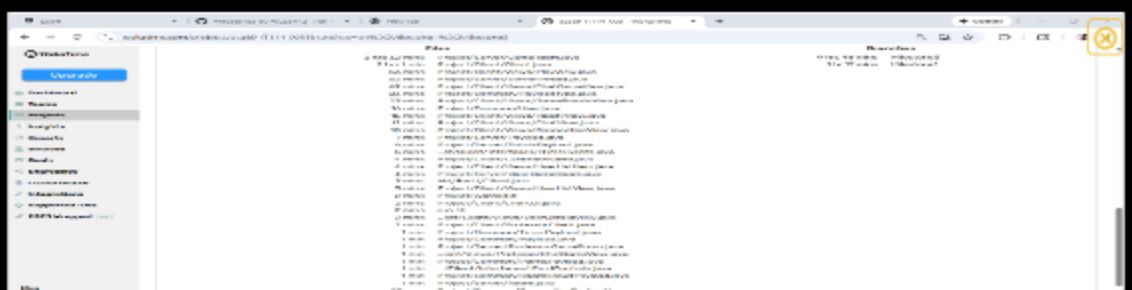


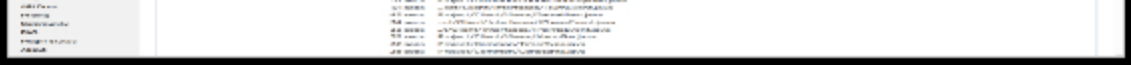
Milestone3 time



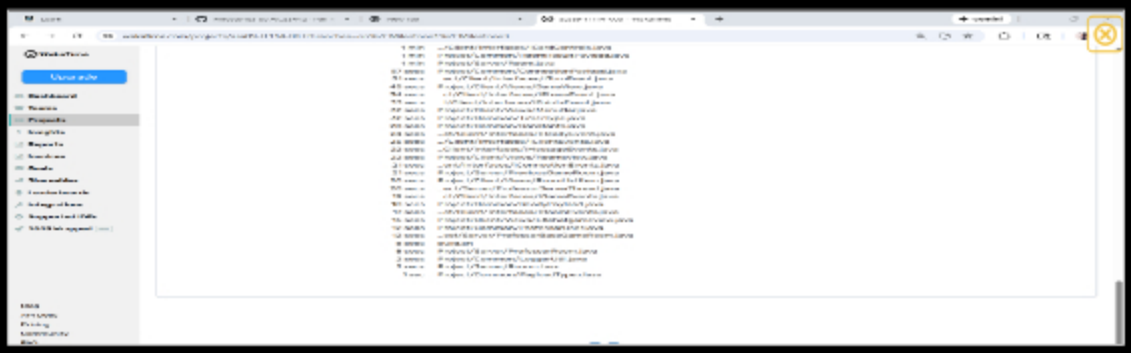
Milestone3 time

overall Project time





overall Project time



overall Project time

Saved: 12/11/2025 11:27:13 PM

Task #3 (0.33 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

Throughout this project, I learned how to create a round-robin based multiplayer game using client-server architecture and synchronize game state across several clients. For example, I used ServerThread.sendGameEvent() to broadcast hangman strikes to all clients. This made sure that everyone saw the same game in real time. I also learned about handling timers for rounds and turns, which helped me manage player actions and enforce turn order correctly. Another important lesson was managing user input and updating UI components.Overall, this project taught me how to structure server logic, handle asynchronous communication, and keep UI updates consistent for multiple players.

Copy

Saved: 12/11/2025 11:31:54 PM

Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

I would say the easiest part was setting up the letter buttons using the Baseline UI files. Once I understood hat each button just needed an action listener to call Client.INSTANCE.sendLetter(), it was straightforward to loop through 'A' to 'Z' and generate all the buttons dynamically. For example, adding the line letterButton.addActionListener(_ -> Client.INSTANCE.sendLetter(letter.charAt(0))); instantly linked each button to sending the correct letter to the server.

Saved: 12/11/2025 11:33:50 PM

Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part of this project was creating the Hangman game logic so that it stayed in sync for multiple clients in real time. For example, handling letter guesses meant updating the blanks array, tracking guessed letters, and increasing the gloabl strikes. Another hurdle was making sure the turn-based logic worked properly with turnOrder and currentTurnClientId, making that only the active player could guess while the others waited. And on a lesser note, just on a personal level, it was a little hard to keep track of all the names of the methods, especially as the methods evolved or were added.

Saved: 12/11/2025 11:37:35 PM